

non-strictly_convex_loss

September 14, 2025

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

[2]: # =====
# Generate collinear data
# =====
m = 50
x1 = np.linspace(0, 10, m)
x2 = 2 * x1 # perfectly collinear
X = np.vstack([x1, x2]).T
y = 3 * x1 + 5 + np.random.randn(m) * 0.5 # small noise

[3]: # =====
# Compute loss J(theta) on a grid
# =====
theta0_vals = np.linspace(0, 10, 50)
theta1_vals = np.linspace(-1, 5, 50)
J_vals = np.zeros((len(theta0_vals), len(theta1_vals)))

for i, t0 in enumerate(theta0_vals):
    for j, t1 in enumerate(theta1_vals):
        theta = np.array([t0, t1])
        J_vals[i, j] = 0.5 * np.mean((X @ theta - y)**2)

[4]: # =====
# 3D plot
# =====
T0, T1 = np.meshgrid(theta0_vals, theta1_vals)
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(T0, T1, J_vals.T, cmap='viridis', alpha=0.9)
ax.set_xlabel('theta0')
ax.set_ylabel('theta1')
ax.set_zlabel('J(theta)')
ax.set_title('Convex but not strictly convex loss (flat valley)')
plt.show()
```

Convex but not strictly convex loss (flat valley)

